Lab5_rs_fmri_LSTM_classification

December 1, 2025

0.1 Using LSTM to analyze resting state networks

This notebook includes code to classify ADHD from healthy controls using LSTM model with rs-fMRI data.

First let's import the relevant libraries

[1]:
```
!pip3 install nilearn
from nilearn import datasets
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import warnings
warnings.filterwarnings("ignore")
plt.style.use('ggplot')

#nilearn - neuroimaging tailored library
from nilearn.input_data import NiftiMapsMasker
from nilearn import plotting

#sklearn - basic ML tools
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_curve
from sklearn import metrics

#keras - for NN models
from keras.models import Model, Sequential
from keras.layers import Input, Dense
from keras.layers import LSTM
from keras import optimizers
from keras.utils import plot_model
from keras import utils
from sklearn.metrics import roc_curve


#scipy- statistical analysis tools
from scipy.stats import ttest_1samp
#from scipy import interp
```

```
Requirement already satisfied: nilearn in
```

```
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(0.10.2)
Requirement already satisfied: joblib>=1.0.0 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from nilearn) (1.4.2)
Requirement already satisfied: lxml in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from nilearn) (5.3.1)
Requirement already satisfied: nibabel>=3.2.0 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from nilearn) (5.1.0)
Requirement already satisfied: numpy>=1.19.0 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from nilearn) (1.26.4)
Requirement already satisfied: packaging in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from nilearn) (24.2)
Requirement already satisfied: pandas>=1.1.5 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from nilearn) (2.3.3)
Requirement already satisfied: requests>=2.25.0 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from nilearn) (2.32.3)
Requirement already satisfied: scikit-learn>=1.0.0 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from nilearn) (1.6.1)
Requirement already satisfied: scipy>=1.6.0 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from nilearn) (1.11.4)
Requirement already satisfied: python-dateutil>=2.8.2 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from pandas>=1.1.5->nilearn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from pandas>=1.1.5->nilearn) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from pandas>=1.1.5->nilearn) (2025.2)
Requirement already satisfied: six>=1.5 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from python-dateutil>=2.8.2->pandas>=1.1.5->nilearn) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from requests>=2.25.0->nilearn) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from requests>=2.25.0->nilearn) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
```

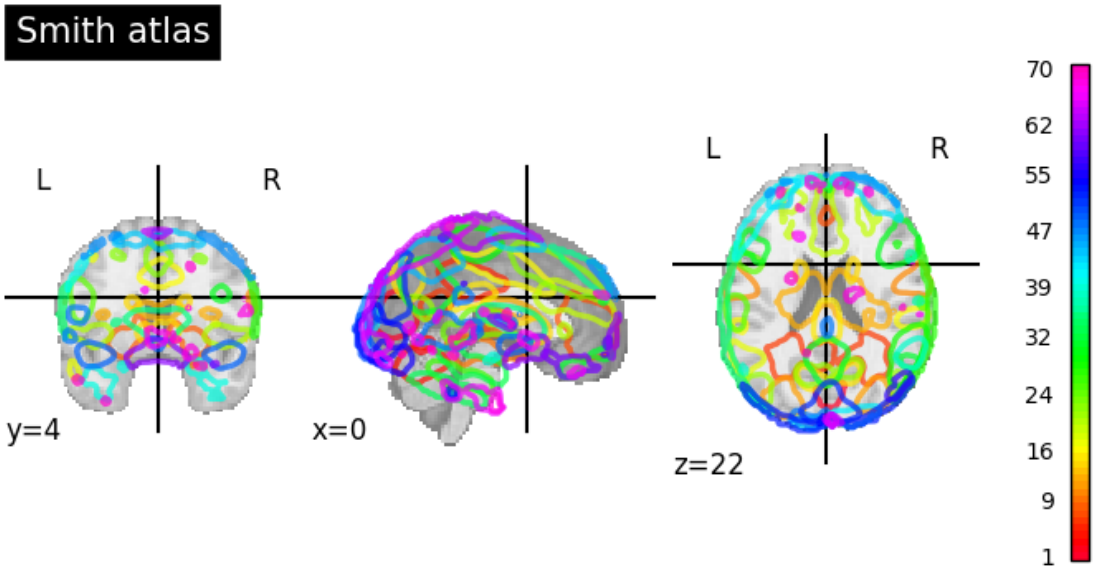
```
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from requests>=2.25.0->nilearn) (2.4.0)
Requirement already satisfied: certifi>=2017.4.17 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from requests>=2.25.0->nilearn) (2025.1.31)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from scikit-learn>=1.0.0->nilearn) (3.6.0)
```

0.2 Preparing data for analysis

Import data, and exctract features

[2]:
```
## load the smith (ICA based) mask
## 'rsn10': 10 ICA maps from the above that matched across task and rest
# http://brainmap.org/pubs/SmithPNAS09.pdf

smith_atlas = datasets.fetch_atlas_smith_2009()
smith_atlas_rs_networks = smith_atlas.rsn70
```

[3]:
```
plotting.plot_prob_atlas(smith_atlas_rs_networks,
                         title='Smith atlas',
                         colorbar=True)
plotting.show()
```



[4]:
```
# Import preprocessed, ready-to-go, datasets
adhd_data=datasets.fetch_adhd(n_subjects=50)
```

[5]:
```
# exploring the header of one image
import nibabel as nib
img=nib.load(adhd_data['func'][0])

print(img.header['dim'])
print(img.header['pixdim'])
```

```
[  4  61  73  61 176   1   1   1]
[-1.  3.  3.  3.  2.  0.  0.  0.]
```

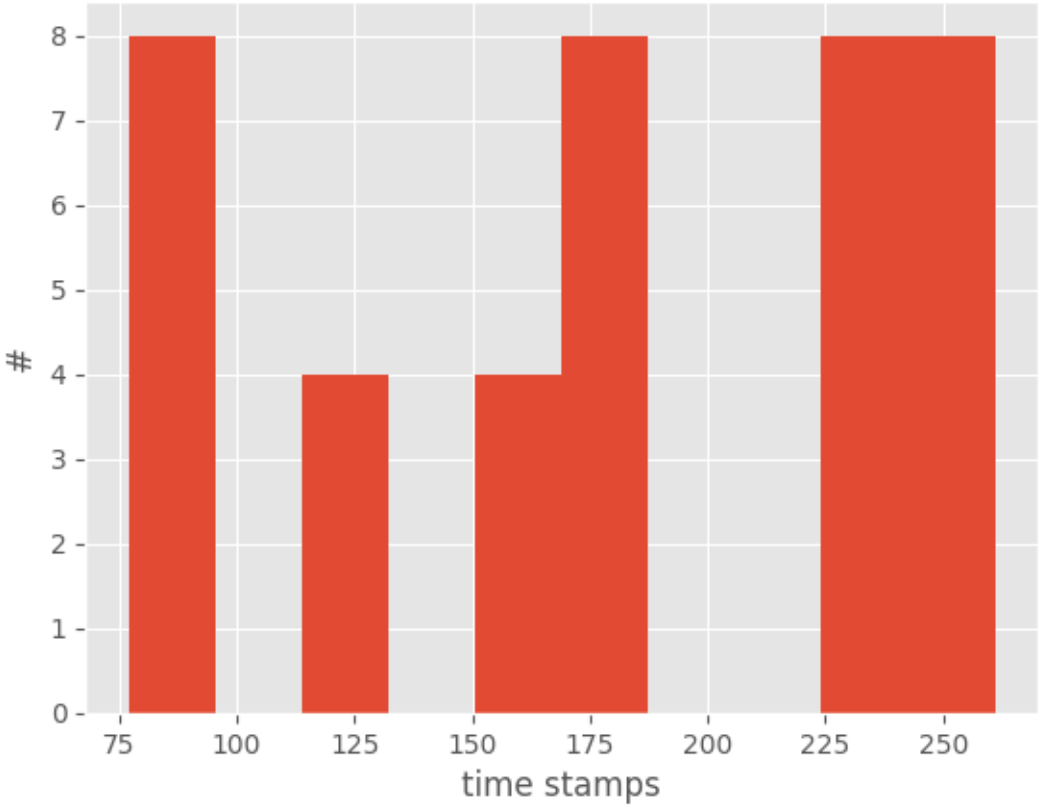
[6]:
```
# Region signals extraction- exctract the values of the ten networks

masker = NiftiMapsMasker(maps_img=smith_atlas_rs_networks,  # Smith stals
                         standardize=True, # centers and norms the time-series
                         memory='nilearn_cache', # cache
                         verbose=0) #do not print verbose
```

[7]:
```
all_subjects_data=[]
labels=[]  # 1 if ADHD, 0 if control

for func_file, confound_file, phenotypic in zip(
        adhd_data.func, adhd_data.confounds, adhd_data.phenotypic):

    time_series = masker.fit_transform(func_file, confounds=confound_file)

    all_subjects_data.append(time_series)
    labels.append(phenotypic['adhd'])
```

[8]:
```
print('N control:' ,labels.count(0))
print('N adhd:' ,labels.count(1))
```

```
N control: 20
N adhd: 20
```

[9]:
```
plt.hist([len(i) for i in all_subjects_data])
plt.title('fMRI dataset length variation')
plt.xlabel('time stamps')
plt.ylabel('#')
plt.show()
```



[10]:
```
# find the longest image
max_len_image=np.max([len(i) for i in all_subjects_data])
```

[15]:
```
# reshape

all_subjects_data_reshaped=[]
for subject_data in all_subjects_data:
  # Padding
  N= max_len_image-len(subject_data)
  padded_array=np.pad(subject_data, ((0, N), (0,0)),
                      'constant', constant_values=(0))
  subject_data=padded_array
  subject_data=np.array(subject_data)
  subject_data.reshape(subject_data.shape[0],subject_data.shape[1],1)
  all_subjects_data_reshaped.append(subject_data)
```

[16]:
```
# shape of data

# 40 subjects
```

```
# 261 time stamps
# 10 netwroks values

np.array(all_subjects_data_reshaped).shape
```

[16]: (40, 261, 70)

[17]:
```
# The data, split between train and test sets.

def get_train_test(X, y, i, verbrose=False):
  '''
  split to train and test and reshape data
  X data
  y labels
  i random state
  '''
  X_train, X_test, y_train, y_test = train_test_split(X,
                                        y, test_size=0.2, random_state=i)



  # Reshapes data to 4D for Hierarchical RNN.
  t_shape=np.array(all_subjects_data_reshaped).shape[1]
  RSN_shape=np.array(all_subjects_data_reshaped).shape[2]

  X_train = np.reshape(X_train, (len(X_train), t_shape, RSN_shape))
  X_test = np.reshape(X_test, (len(X_test), t_shape, RSN_shape))

  X_train = X_train.astype('float32')
  X_test = X_test.astype('float32')

  if verbrose:
    print(X_train.shape[0], 'train samples')
    print(X_test.shape[0], 'test samples')

  # Converts class vectors to binary class matrices.
  y_train = utils.to_categorical(y_train, 2)
  y_test = utils.to_categorical(y_test, 2)

  return X_train, X_test, y_train, y_test
```

0.3 Build the LSTM model

[18]:
```
# create the model

model = Sequential()
```

```
# LSTM layers -
# Long Short-Term Memory layer - Hochreiter 1997.
t_shape=np.array(all_subjects_data_reshaped).shape[1]
RSN_shape=np.array(all_subjects_data_reshaped).shape[2]
print(f"Input shape: ({t_shape}, {RSN_shape})")

# NOTE: Using only 2 LSTM layers instead of 4, with smaller units
model.add(LSTM(units=32,
               return_sequences=True,
               input_shape=(t_shape, RSN_shape)))

model.add(LSTM(units=16,
               return_sequences=False))

model.add(Dense(units=2,
               activation="softmax")) # Changed to softmax for binary␣
 ↪classification

model.compile(loss='categorical_crossentropy',
               optimizer=optimizers.Adam(learning_rate=0.001),
               metrics=['accuracy'])

print(model.summary())
```

```
Input shape: (261, 70)
```

```
2025-11-30 23:59:18.907584: I metal_plugin/src/device/metal_device.cc:1154]
Metal device set to: Apple M1
2025-11-30 23:59:18.908051: I metal_plugin/src/device/metal_device.cc:296]
systemMemory: 8.00 GB
2025-11-30 23:59:18.908056: I metal_plugin/src/device/metal_device.cc:313]
maxCacheSize: 2.67 GB
2025-11-30 23:59:18.908685: I
tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:305]
Could not identify NUMA node of platform GPU ID 0, defaulting to 0. Your kernel
may not have been built with NUMA support.
2025-11-30 23:59:18.909516: I
tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:271]
Created TensorFlow device (/job:localhost/replica:0/task:0/device:GPU:0 with 0
MB memory) -> physical PluggableDevice (device: 0, name: METAL, pci bus id:
<undefined>)
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 261, 32)	13,184

```
 lstm_1 (LSTM)                   (None, 16)                    3,136

 dense (Dense)                   (None, 2)                        34


 Total params: 16,354 (63.88 KB)

 Trainable params: 16,354 (63.88 KB)

 Non-trainable params: 0 (0.00 B)

None
```

[19]:
```
# plot_model(model, show_shapes=True, show_layer_names=True)
```

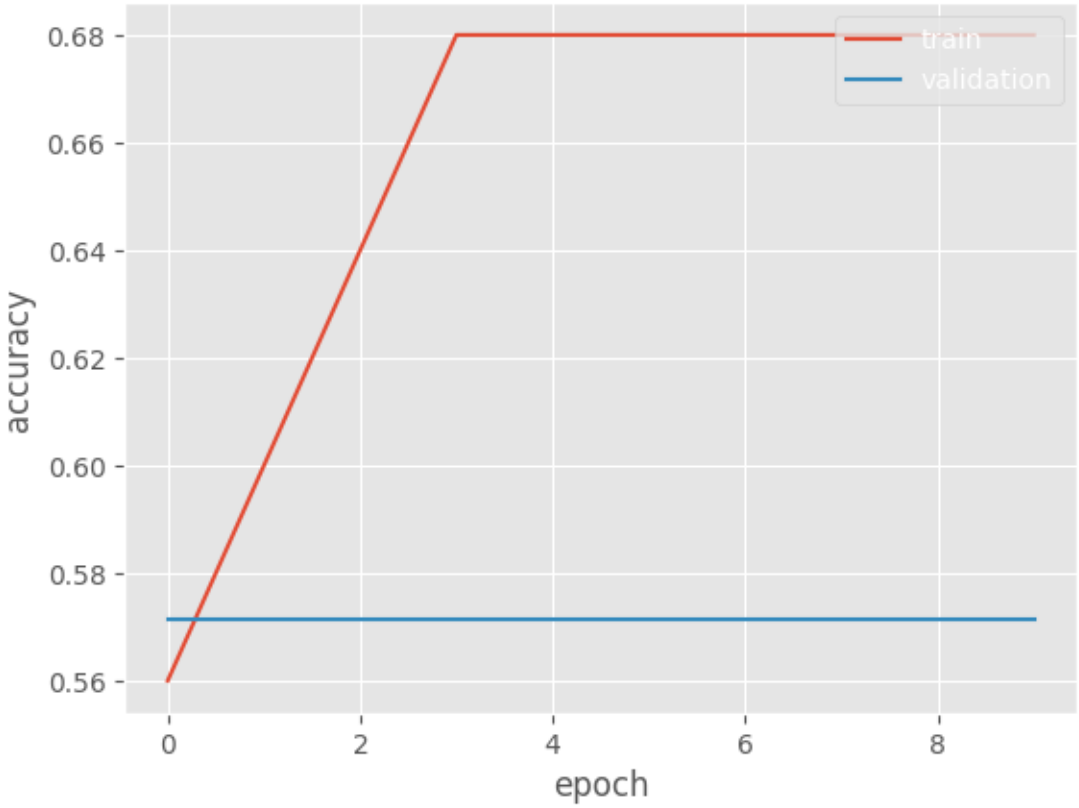
0.4 Train the LSTM model

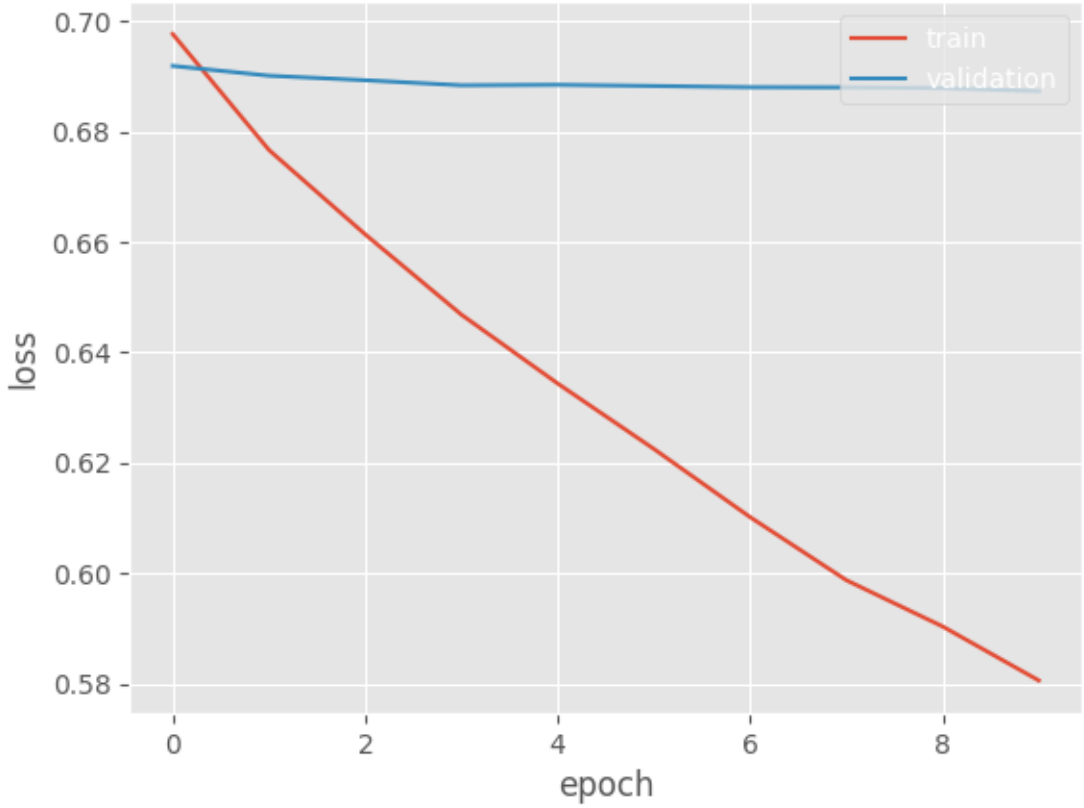
[20]:
```
X_train, X_test, y_train, y_test = get_train_test(all_subjects_data_reshaped,
labels, i=8, verbrose=True)

print(f"\nStarting training... Each epoch should take ~5-10 seconds on CPU")
print(f"Training data shape: {X_train.shape}")

# Reduced epochs to 10 for faster initial testing, increased batch_size
import time
start_time = time.time()

history = model.fit(X_train, y_train,
                    validation_split=0.2,
                    epochs=10, # Reduced from 30 for faster testing
                    batch_size=16, # Increased from 8
                    verbose=1)

elapsed_time = time.time() - start_time
print(f"\n  Training completed in {elapsed_time:.1f} seconds ({elapsed_time/10:.
 ↪1f}s per epoch)")

# summarize history for accuracy
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'validation'], loc='upper right')
```

```
plt.show()

# summarize history for loss
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'validation'], loc='upper right')
plt.show()
```

```
32 train samples
8 test samples

Starting training… Each epoch should take ~5-10 seconds on CPU
Training data shape: (32, 261, 70)
Epoch 1/10

2025-12-01 00:00:15.167503: I
tensorflow/core/grappler/optimizers/custom_graph_optimizer_registry.cc:117]
Plugin optimizer for device_type GPU is enabled.

2/2             5s 2s/step -
accuracy: 0.5608 - loss: 0.6982 - val_accuracy: 0.5714 - val_loss: 0.6919
Epoch 2/10
2/2             0s 92ms/step -
accuracy: 0.5875 - loss: 0.6800 - val_accuracy: 0.5714 - val_loss: 0.6901
Epoch 3/10
2/2             0s 94ms/step -
accuracy: 0.6142 - loss: 0.6621 - val_accuracy: 0.5714 - val_loss: 0.6893
Epoch 4/10
2/2             0s 91ms/step -
accuracy: 0.7033 - loss: 0.6478 - val_accuracy: 0.5714 - val_loss: 0.6884
Epoch 5/10
2/2             0s 88ms/step -
accuracy: 0.7033 - loss: 0.6341 - val_accuracy: 0.5714 - val_loss: 0.6885
Epoch 6/10
2/2             0s 89ms/step -
accuracy: 0.7033 - loss: 0.6176 - val_accuracy: 0.5714 - val_loss: 0.6882
Epoch 7/10
2/2             0s 92ms/step -
accuracy: 0.7033 - loss: 0.6098 - val_accuracy: 0.5714 - val_loss: 0.6880
Epoch 8/10
2/2             0s 94ms/step -
accuracy: 0.6617 - loss: 0.6035 - val_accuracy: 0.5714 - val_loss: 0.6880
Epoch 9/10
2/2             0s 152ms/step -
accuracy: 0.7033 - loss: 0.5824 - val_accuracy: 0.5714 - val_loss: 0.6879
Epoch 10/10
```

```
2/2             0s 89ms/step -
accuracy: 0.7033 - loss: 0.5619 - val_accuracy: 0.5714 - val_loss: 0.6873

 Training completed in 6.3 seconds (0.6s per epoch)
```





0.5 Evaluate the LSTM model

[21]:
```
from sklearn.metrics import accuracy_score

def boostrapping_hypothesis_testing(X_train, y_train, X_test, y_test,
                                    n_iterations=5, n_epochs=50):

  '''
  hypothesis testing function
  X_train, y_train, X_test, y_test- the data
  n_iterations- number of bootdtaping iterations
  n_epochs - number of epochs for model's training
  '''

  accuracy=[] ## model accuracy
  roc_msrmnts_fpr=[] ## false positive rate
  roc_msrmnts_tpr=[] ## true positive rate

  # run bootstrap
```

```
  for i in range(n_iterations):
    # prepare train and test sets
    X_train, X_test, y_train, y_test=get_train_test(all_subjects_data_reshaped,
                                                    labels, i=i, verbrose=False)
    # fit model
    print('fitting..')
    model.fit(X_train, y_train, validation_split=0.2, epochs=n_epochs)

    # evaluate model
    print('evaluating..')
    y_pred=model.predict(X_test)
    y_test_1d=[i[0] for i in y_test]
    y_pred_1d=[1.0 if i[0]>.5 else 0.0 for i in y_pred]

    fpr, tpr, _ = roc_curve(y_test_1d, y_pred_1d)

    acc_score = accuracy_score(y_test_1d, y_pred_1d)

    accuracy.append(acc_score)
    roc_msrmnts_fpr.append(fpr)
    roc_msrmnts_tpr.append(tpr)

  return accuracy, roc_msrmnts_fpr, roc_msrmnts_tpr



accuracy, roc_msrmnts_fpr, roc_msrmnts_tpr  =␣
 ↪boostrapping_hypothesis_testing(X_train, y_train, X_test, y_test)
```

```
fitting..
Epoch 1/50
1/1             0s 480ms/step -
accuracy: 0.5200 - loss: 0.6244 - val_accuracy: 0.8571 - val_loss: 0.5577
Epoch 2/50
1/1             0s 105ms/step -
accuracy: 0.5200 - loss: 0.6218 - val_accuracy: 0.8571 - val_loss: 0.5550
Epoch 3/50
1/1             0s 107ms/step -
accuracy: 0.5200 - loss: 0.6184 - val_accuracy: 0.8571 - val_loss: 0.5540
Epoch 4/50
1/1             0s 110ms/step -
accuracy: 0.5600 - loss: 0.6141 - val_accuracy: 0.8571 - val_loss: 0.5543
Epoch 5/50
1/1             0s 111ms/step -
accuracy: 0.5600 - loss: 0.6091 - val_accuracy: 0.8571 - val_loss: 0.5558
Epoch 6/50
1/1             0s 109ms/step -
accuracy: 0.5600 - loss: 0.6036 - val_accuracy: 0.8571 - val_loss: 0.5584
```

```
Epoch 7/50
1/1             0s 105ms/step -
accuracy: 0.5600 - loss: 0.5977 - val_accuracy: 0.8571 - val_loss: 0.5620
Epoch 8/50
1/1             0s 104ms/step -
accuracy: 0.5600 - loss: 0.5916 - val_accuracy: 0.8571 - val_loss: 0.5664
Epoch 9/50
1/1             0s 107ms/step -
accuracy: 0.5600 - loss: 0.5854 - val_accuracy: 0.8571 - val_loss: 0.5717
Epoch 10/50
1/1             0s 104ms/step -
accuracy: 0.5600 - loss: 0.5793 - val_accuracy: 0.8571 - val_loss: 0.5776
Epoch 11/50
1/1             0s 104ms/step -
accuracy: 0.5600 - loss: 0.5734 - val_accuracy: 0.8571 - val_loss: 0.5844
Epoch 12/50
1/1             0s 98ms/step -
accuracy: 0.5600 - loss: 0.5677 - val_accuracy: 0.8571 - val_loss: 0.5919
Epoch 13/50
1/1             0s 107ms/step -
accuracy: 0.6000 - loss: 0.5622 - val_accuracy: 0.7143 - val_loss: 0.6002
Epoch 14/50
1/1             0s 113ms/step -
accuracy: 0.6800 - loss: 0.5571 - val_accuracy: 0.7143 - val_loss: 0.6094
Epoch 15/50
1/1             0s 106ms/step -
accuracy: 0.6800 - loss: 0.5522 - val_accuracy: 0.5714 - val_loss: 0.6195
Epoch 16/50
1/1             0s 104ms/step -
accuracy: 0.6800 - loss: 0.5476 - val_accuracy: 0.2857 - val_loss: 0.6307
Epoch 17/50
1/1             0s 106ms/step -
accuracy: 0.6800 - loss: 0.5434 - val_accuracy: 0.2857 - val_loss: 0.6430
Epoch 18/50
1/1             0s 106ms/step -
accuracy: 0.6800 - loss: 0.5397 - val_accuracy: 0.2857 - val_loss: 0.6564
Epoch 19/50
1/1             0s 103ms/step -
accuracy: 0.6800 - loss: 0.5363 - val_accuracy: 0.2857 - val_loss: 0.6709
Epoch 20/50
1/1             0s 112ms/step -
accuracy: 0.6800 - loss: 0.5336 - val_accuracy: 0.2857 - val_loss: 0.6862
Epoch 21/50
1/1             0s 152ms/step -
accuracy: 0.6800 - loss: 0.5314 - val_accuracy: 0.2857 - val_loss: 0.7018
Epoch 22/50
1/1             0s 118ms/step -
accuracy: 0.6800 - loss: 0.5298 - val_accuracy: 0.2857 - val_loss: 0.7169
```

```
Epoch 23/50
1/1             0s 109ms/step -
accuracy: 0.6800 - loss: 0.5287 - val_accuracy: 0.2857 - val_loss: 0.7304
Epoch 24/50
1/1             0s 105ms/step -
accuracy: 0.6800 - loss: 0.5280 - val_accuracy: 0.2857 - val_loss: 0.7410
Epoch 25/50
1/1             0s 109ms/step -
accuracy: 0.6800 - loss: 0.5274 - val_accuracy: 0.2857 - val_loss: 0.7477
Epoch 26/50
1/1             0s 174ms/step -
accuracy: 0.6800 - loss: 0.5265 - val_accuracy: 0.2857 - val_loss: 0.7498
Epoch 27/50
1/1             0s 106ms/step -
accuracy: 0.6800 - loss: 0.5252 - val_accuracy: 0.2857 - val_loss: 0.7473
Epoch 28/50
1/1             0s 105ms/step -
accuracy: 0.6800 - loss: 0.5233 - val_accuracy: 0.2857 - val_loss: 0.7409
Epoch 29/50
1/1             0s 106ms/step -
accuracy: 0.6800 - loss: 0.5210 - val_accuracy: 0.2857 - val_loss: 0.7313
Epoch 30/50
1/1             0s 103ms/step -
accuracy: 0.6800 - loss: 0.5184 - val_accuracy: 0.2857 - val_loss: 0.7195
Epoch 31/50
1/1             0s 107ms/step -
accuracy: 0.6800 - loss: 0.5156 - val_accuracy: 0.2857 - val_loss: 0.7064
Epoch 32/50
1/1             0s 113ms/step -
accuracy: 0.6800 - loss: 0.5129 - val_accuracy: 0.2857 - val_loss: 0.6928
Epoch 33/50
1/1             0s 106ms/step -
accuracy: 0.6800 - loss: 0.5102 - val_accuracy: 0.2857 - val_loss: 0.6793
Epoch 34/50
1/1             0s 157ms/step -
accuracy: 0.6800 - loss: 0.5077 - val_accuracy: 0.2857 - val_loss: 0.6662
Epoch 35/50
1/1             0s 113ms/step -
accuracy: 0.6800 - loss: 0.5052 - val_accuracy: 0.2857 - val_loss: 0.6540
Epoch 36/50
1/1             0s 110ms/step -
accuracy: 0.6800 - loss: 0.5027 - val_accuracy: 0.2857 - val_loss: 0.6428
Epoch 37/50
1/1             0s 114ms/step -
accuracy: 0.6800 - loss: 0.5001 - val_accuracy: 0.2857 - val_loss: 0.6328
Epoch 38/50
1/1             0s 114ms/step -
accuracy: 0.6800 - loss: 0.4974 - val_accuracy: 0.2857 - val_loss: 0.6245
```

```
Epoch 39/50
1/1             0s 111ms/step -
accuracy: 0.6800 - loss: 0.4946 - val_accuracy: 0.2857 - val_loss: 0.6181
Epoch 40/50
1/1             0s 113ms/step -
accuracy: 0.6800 - loss: 0.4918 - val_accuracy: 0.2857 - val_loss: 0.6143
Epoch 41/50
1/1             0s 112ms/step -
accuracy: 0.6800 - loss: 0.4890 - val_accuracy: 0.2857 - val_loss: 0.6138
Epoch 42/50
1/1             0s 108ms/step -
accuracy: 0.6800 - loss: 0.4864 - val_accuracy: 0.2857 - val_loss: 0.6175
Epoch 43/50
1/1             0s 103ms/step -
accuracy: 0.6800 - loss: 0.4840 - val_accuracy: 0.2857 - val_loss: 0.6263
Epoch 44/50
1/1             0s 99ms/step -
accuracy: 0.6800 - loss: 0.4817 - val_accuracy: 0.2857 - val_loss: 0.6414
Epoch 45/50
1/1             0s 95ms/step -
accuracy: 0.6800 - loss: 0.4797 - val_accuracy: 0.2857 - val_loss: 0.6633
Epoch 46/50
1/1             0s 99ms/step -
accuracy: 0.6800 - loss: 0.4780 - val_accuracy: 0.2857 - val_loss: 0.6911
Epoch 47/50
1/1             0s 145ms/step -
accuracy: 0.6800 - loss: 0.4763 - val_accuracy: 0.2857 - val_loss: 0.7213
Epoch 48/50
1/1             0s 114ms/step -
accuracy: 0.6800 - loss: 0.4722 - val_accuracy: 0.2857 - val_loss: 0.7580
Epoch 49/50
1/1             0s 110ms/step -
accuracy: 0.6800 - loss: 0.4591 - val_accuracy: 0.1429 - val_loss: 0.8291
Epoch 50/50
1/1             0s 105ms/step -
accuracy: 0.7200 - loss: 0.4302 - val_accuracy: 0.2857 - val_loss: 0.9755
evaluating..
1/1             0s 296ms/step
fitting..
Epoch 1/50
1/1             0s 112ms/step -
accuracy: 0.6000 - loss: 0.5607 - val_accuracy: 0.8571 - val_loss: 0.3737
Epoch 2/50
1/1             0s 107ms/step -
accuracy: 0.7200 - loss: 0.4848 - val_accuracy: 0.8571 - val_loss: 0.3717
Epoch 3/50
1/1             0s 120ms/step -
accuracy: 0.7200 - loss: 0.4766 - val_accuracy: 0.8571 - val_loss: 0.3721
```

```
Epoch 4/50
1/1             0s 110ms/step -
accuracy: 0.6800 - loss: 0.4738 - val_accuracy: 0.8571 - val_loss: 0.3725
Epoch 5/50
1/1             0s 107ms/step -
accuracy: 0.7200 - loss: 0.4641 - val_accuracy: 0.8571 - val_loss: 0.3722
Epoch 6/50
1/1             0s 111ms/step -
accuracy: 0.7200 - loss: 0.4519 - val_accuracy: 0.8571 - val_loss: 0.3716
Epoch 7/50
1/1             0s 163ms/step -
accuracy: 0.7200 - loss: 0.4408 - val_accuracy: 0.8571 - val_loss: 0.3710
Epoch 8/50
1/1             0s 115ms/step -
accuracy: 0.7200 - loss: 0.4290 - val_accuracy: 0.8571 - val_loss: 0.3709
Epoch 9/50
1/1             0s 110ms/step -
accuracy: 0.7600 - loss: 0.4126 - val_accuracy: 0.8571 - val_loss: 0.3713
Epoch 10/50
1/1             0s 110ms/step -
accuracy: 0.8000 - loss: 0.3873 - val_accuracy: 0.8571 - val_loss: 0.3727
Epoch 11/50
1/1             0s 110ms/step -
accuracy: 0.8400 - loss: 0.3560 - val_accuracy: 0.8571 - val_loss: 0.3758
Epoch 12/50
1/1             0s 114ms/step -
accuracy: 0.9200 - loss: 0.3199 - val_accuracy: 0.7143 - val_loss: 0.3845
Epoch 13/50
1/1             0s 105ms/step -
accuracy: 0.8800 - loss: 0.2654 - val_accuracy: 0.8571 - val_loss: 0.4484
Epoch 14/50
1/1             0s 106ms/step -
accuracy: 0.8800 - loss: 0.2830 - val_accuracy: 0.8571 - val_loss: 0.5305
Epoch 15/50
1/1             0s 100ms/step -
accuracy: 0.8800 - loss: 0.2509 - val_accuracy: 0.8571 - val_loss: 0.4806
Epoch 16/50
1/1             0s 102ms/step -
accuracy: 0.9200 - loss: 0.2234 - val_accuracy: 0.8571 - val_loss: 0.3142
Epoch 17/50
1/1             0s 106ms/step -
accuracy: 0.9200 - loss: 0.2093 - val_accuracy: 1.0000 - val_loss: 0.1629
Epoch 18/50
1/1             0s 101ms/step -
accuracy: 0.9200 - loss: 0.2047 - val_accuracy: 1.0000 - val_loss: 0.1088
Epoch 19/50
1/1             0s 109ms/step -
accuracy: 0.9200 - loss: 0.2067 - val_accuracy: 1.0000 - val_loss: 0.0855
```

```
Epoch 20/50
1/1             0s 102ms/step -
accuracy: 0.9200 - loss: 0.2114 - val_accuracy: 1.0000 - val_loss: 0.0738
Epoch 21/50
1/1             0s 147ms/step -
accuracy: 0.9200 - loss: 0.2161 - val_accuracy: 1.0000 - val_loss: 0.0680
Epoch 22/50
1/1             0s 108ms/step -
accuracy: 0.9200 - loss: 0.2193 - val_accuracy: 1.0000 - val_loss: 0.0658
Epoch 23/50
1/1             0s 101ms/step -
accuracy: 0.9200 - loss: 0.2204 - val_accuracy: 1.0000 - val_loss: 0.0663
Epoch 24/50
1/1             0s 104ms/step -
accuracy: 0.9200 - loss: 0.2198 - val_accuracy: 1.0000 - val_loss: 0.0688
Epoch 25/50
1/1             0s 107ms/step -
accuracy: 0.9200 - loss: 0.2177 - val_accuracy: 1.0000 - val_loss: 0.0732
Epoch 26/50
1/1             0s 106ms/step -
accuracy: 0.9200 - loss: 0.2147 - val_accuracy: 1.0000 - val_loss: 0.0794
Epoch 27/50
1/1             0s 102ms/step -
accuracy: 0.9200 - loss: 0.2114 - val_accuracy: 1.0000 - val_loss: 0.0872
Epoch 28/50
1/1             0s 99ms/step -
accuracy: 0.9200 - loss: 0.2084 - val_accuracy: 1.0000 - val_loss: 0.0969
Epoch 29/50
1/1             0s 97ms/step -
accuracy: 0.9200 - loss: 0.2059 - val_accuracy: 1.0000 - val_loss: 0.1091
Epoch 30/50
1/1             0s 106ms/step -
accuracy: 0.9200 - loss: 0.2044 - val_accuracy: 1.0000 - val_loss: 0.1244
Epoch 31/50
1/1             0s 103ms/step -
accuracy: 0.9200 - loss: 0.2036 - val_accuracy: 1.0000 - val_loss: 0.1393
Epoch 32/50
1/1             0s 107ms/step -
accuracy: 0.9200 - loss: 0.2035 - val_accuracy: 1.0000 - val_loss: 0.1489
Epoch 33/50
1/1             0s 102ms/step -
accuracy: 0.9200 - loss: 0.2038 - val_accuracy: 1.0000 - val_loss: 0.1524
Epoch 34/50
1/1             0s 109ms/step -
accuracy: 0.9200 - loss: 0.2042 - val_accuracy: 1.0000 - val_loss: 0.1531
Epoch 35/50
1/1             0s 103ms/step -
accuracy: 0.9200 - loss: 0.2045 - val_accuracy: 1.0000 - val_loss: 0.1539
```

```
Epoch 36/50
1/1             0s 142ms/step -
accuracy: 0.9200 - loss: 0.2044 - val_accuracy: 1.0000 - val_loss: 0.1547
Epoch 37/50
1/1             0s 101ms/step -
accuracy: 0.9200 - loss: 0.2040 - val_accuracy: 1.0000 - val_loss: 0.1549
Epoch 38/50
1/1             0s 100ms/step -
accuracy: 0.9200 - loss: 0.2030 - val_accuracy: 1.0000 - val_loss: 0.1538
Epoch 39/50
1/1             0s 105ms/step -
accuracy: 0.9200 - loss: 0.2015 - val_accuracy: 1.0000 - val_loss: 0.1513
Epoch 40/50
1/1             0s 107ms/step -
accuracy: 0.9200 - loss: 0.1995 - val_accuracy: 1.0000 - val_loss: 0.1479
Epoch 41/50
1/1             0s 105ms/step -
accuracy: 0.9200 - loss: 0.1971 - val_accuracy: 1.0000 - val_loss: 0.1440
Epoch 42/50
1/1             0s 103ms/step -
accuracy: 0.9200 - loss: 0.1942 - val_accuracy: 1.0000 - val_loss: 0.1408
Epoch 43/50
1/1             0s 109ms/step -
accuracy: 0.9200 - loss: 0.1900 - val_accuracy: 1.0000 - val_loss: 0.1409
Epoch 44/50
1/1             0s 103ms/step -
accuracy: 0.9200 - loss: 0.1802 - val_accuracy: 1.0000 - val_loss: 0.1717
Epoch 45/50
1/1             0s 106ms/step -
accuracy: 0.9600 - loss: 0.1337 - val_accuracy: 0.8571 - val_loss: 0.6107
Epoch 46/50
1/1             0s 103ms/step -
accuracy: 0.9600 - loss: 0.1292 - val_accuracy: 0.8571 - val_loss: 0.8645
Epoch 47/50
1/1             0s 109ms/step -
accuracy: 0.9600 - loss: 0.1241 - val_accuracy: 0.8571 - val_loss: 0.8607
Epoch 48/50
1/1             0s 120ms/step -
accuracy: 0.9600 - loss: 0.1195 - val_accuracy: 0.8571 - val_loss: 0.8539
Epoch 49/50
1/1             0s 108ms/step -
accuracy: 0.9600 - loss: 0.1160 - val_accuracy: 0.8571 - val_loss: 0.8493
Epoch 50/50
1/1             0s 138ms/step -
accuracy: 0.9600 - loss: 0.1130 - val_accuracy: 0.8571 - val_loss: 0.8502
evaluating..
1/1             0s 32ms/step
fitting..
```

```
Epoch 1/50
1/1             0s 127ms/step -
accuracy: 0.9200 - loss: 0.3398 - val_accuracy: 1.0000 - val_loss: 0.0526
Epoch 2/50
1/1             0s 102ms/step -
accuracy: 0.9200 - loss: 0.3240 - val_accuracy: 1.0000 - val_loss: 0.0789
Epoch 3/50
1/1             0s 124ms/step -
accuracy: 0.9600 - loss: 0.2813 - val_accuracy: 1.0000 - val_loss: 0.0451
Epoch 4/50
1/1             0s 104ms/step -
accuracy: 0.9600 - loss: 0.2461 - val_accuracy: 1.0000 - val_loss: 0.0261
Epoch 5/50
1/1             0s 106ms/step -
accuracy: 0.9600 - loss: 0.2294 - val_accuracy: 1.0000 - val_loss: 0.0199
Epoch 6/50
1/1             0s 104ms/step -
accuracy: 0.9600 - loss: 0.2200 - val_accuracy: 1.0000 - val_loss: 0.0173
Epoch 7/50
1/1             0s 102ms/step -
accuracy: 0.9600 - loss: 0.2128 - val_accuracy: 1.0000 - val_loss: 0.0164
Epoch 8/50
1/1             0s 105ms/step -
accuracy: 0.9600 - loss: 0.2064 - val_accuracy: 1.0000 - val_loss: 0.0167
Epoch 9/50
1/1             0s 103ms/step -
accuracy: 0.9600 - loss: 0.2005 - val_accuracy: 1.0000 - val_loss: 0.0181
Epoch 10/50
1/1             0s 104ms/step -
accuracy: 0.9600 - loss: 0.1949 - val_accuracy: 1.0000 - val_loss: 0.0207
Epoch 11/50
1/1             0s 102ms/step -
accuracy: 0.9600 - loss: 0.1895 - val_accuracy: 1.0000 - val_loss: 0.0247
Epoch 12/50
1/1             0s 203ms/step -
accuracy: 0.9600 - loss: 0.1843 - val_accuracy: 1.0000 - val_loss: 0.0303
Epoch 13/50
1/1             0s 108ms/step -
accuracy: 0.9600 - loss: 0.1792 - val_accuracy: 1.0000 - val_loss: 0.0377
Epoch 14/50
1/1             0s 108ms/step -
accuracy: 0.9600 - loss: 0.1744 - val_accuracy: 1.0000 - val_loss: 0.0469
Epoch 15/50
1/1             0s 103ms/step -
accuracy: 0.9600 - loss: 0.1697 - val_accuracy: 1.0000 - val_loss: 0.0576
Epoch 16/50
1/1             0s 108ms/step -
accuracy: 0.9600 - loss: 0.1653 - val_accuracy: 1.0000 - val_loss: 0.0693
```

```
Epoch 17/50
1/1             0s 102ms/step -
accuracy: 0.9600 - loss: 0.1612 - val_accuracy: 1.0000 - val_loss: 0.0818
Epoch 18/50
1/1             0s 111ms/step -
accuracy: 0.9600 - loss: 0.1575 - val_accuracy: 1.0000 - val_loss: 0.0947
Epoch 19/50
1/1             0s 115ms/step -
accuracy: 0.9600 - loss: 0.1542 - val_accuracy: 1.0000 - val_loss: 0.1077
Epoch 20/50
1/1             0s 112ms/step -
accuracy: 0.9600 - loss: 0.1513 - val_accuracy: 0.8571 - val_loss: 0.1207
Epoch 21/50
1/1             0s 115ms/step -
accuracy: 0.9600 - loss: 0.1489 - val_accuracy: 0.8571 - val_loss: 0.1333
Epoch 22/50
1/1             0s 105ms/step -
accuracy: 0.9600 - loss: 0.1469 - val_accuracy: 0.8571 - val_loss: 0.1455
Epoch 23/50
1/1             0s 108ms/step -
accuracy: 0.9600 - loss: 0.1453 - val_accuracy: 0.8571 - val_loss: 0.1571
Epoch 24/50
1/1             0s 110ms/step -
accuracy: 0.9600 - loss: 0.1441 - val_accuracy: 0.8571 - val_loss: 0.1681
Epoch 25/50
1/1             0s 145ms/step -
accuracy: 0.9600 - loss: 0.1433 - val_accuracy: 0.8571 - val_loss: 0.1784
Epoch 26/50
1/1             0s 102ms/step -
accuracy: 0.9600 - loss: 0.1428 - val_accuracy: 0.8571 - val_loss: 0.1881
Epoch 27/50
1/1             0s 112ms/step -
accuracy: 0.9600 - loss: 0.1426 - val_accuracy: 0.8571 - val_loss: 0.1973
Epoch 28/50
1/1             0s 117ms/step -
accuracy: 0.9600 - loss: 0.1426 - val_accuracy: 0.8571 - val_loss: 0.2059
Epoch 29/50
1/1             0s 110ms/step -
accuracy: 0.9600 - loss: 0.1427 - val_accuracy: 0.8571 - val_loss: 0.2141
Epoch 30/50
1/1             0s 116ms/step -
accuracy: 0.9600 - loss: 0.1429 - val_accuracy: 0.8571 - val_loss: 0.2219
Epoch 31/50
1/1             0s 112ms/step -
accuracy: 0.9600 - loss: 0.1431 - val_accuracy: 0.8571 - val_loss: 0.2295
Epoch 32/50
1/1             0s 108ms/step -
accuracy: 0.9600 - loss: 0.1434 - val_accuracy: 0.8571 - val_loss: 0.2368
```

```
Epoch 33/50
1/1             0s 108ms/step -
accuracy: 0.9600 - loss: 0.1436 - val_accuracy: 0.8571 - val_loss: 0.2439
Epoch 34/50
1/1             0s 107ms/step -
accuracy: 0.9600 - loss: 0.1437 - val_accuracy: 0.8571 - val_loss: 0.2510
Epoch 35/50
1/1             0s 109ms/step -
accuracy: 0.9600 - loss: 0.1438 - val_accuracy: 0.8571 - val_loss: 0.2580
Epoch 36/50
1/1             0s 118ms/step -
accuracy: 0.9600 - loss: 0.1438 - val_accuracy: 0.8571 - val_loss: 0.2651
Epoch 37/50
1/1             0s 116ms/step -
accuracy: 0.9600 - loss: 0.1438 - val_accuracy: 0.8571 - val_loss: 0.2721
Epoch 38/50
1/1             0s 200ms/step -
accuracy: 0.9600 - loss: 0.1436 - val_accuracy: 0.8571 - val_loss: 0.2793
Epoch 39/50
1/1             0s 125ms/step -
accuracy: 0.9600 - loss: 0.1434 - val_accuracy: 0.8571 - val_loss: 0.2865
Epoch 40/50
1/1             0s 122ms/step -
accuracy: 0.9600 - loss: 0.1432 - val_accuracy: 0.8571 - val_loss: 0.2938
Epoch 41/50
1/1             0s 120ms/step -
accuracy: 0.9600 - loss: 0.1430 - val_accuracy: 0.8571 - val_loss: 0.3011
Epoch 42/50
1/1             0s 101ms/step -
accuracy: 0.9600 - loss: 0.1427 - val_accuracy: 0.8571 - val_loss: 0.3085
Epoch 43/50
1/1             0s 111ms/step -
accuracy: 0.9600 - loss: 0.1425 - val_accuracy: 0.8571 - val_loss: 0.3159
Epoch 44/50
1/1             0s 112ms/step -
accuracy: 0.9600 - loss: 0.1422 - val_accuracy: 0.8571 - val_loss: 0.3234
Epoch 45/50
1/1             0s 102ms/step -
accuracy: 0.9600 - loss: 0.1420 - val_accuracy: 0.8571 - val_loss: 0.3309
Epoch 46/50
1/1             0s 108ms/step -
accuracy: 0.9600 - loss: 0.1418 - val_accuracy: 0.8571 - val_loss: 0.3384
Epoch 47/50
1/1             0s 100ms/step -
accuracy: 0.9600 - loss: 0.1416 - val_accuracy: 0.8571 - val_loss: 0.3458
Epoch 48/50
1/1             0s 102ms/step -
accuracy: 0.9600 - loss: 0.1414 - val_accuracy: 0.8571 - val_loss: 0.3532
```

```
Epoch 49/50
1/1             0s 104ms/step -
accuracy: 0.9600 - loss: 0.1413 - val_accuracy: 0.8571 - val_loss: 0.3606
Epoch 50/50
1/1             0s 147ms/step -
accuracy: 0.9600 - loss: 0.1412 - val_accuracy: 0.8571 - val_loss: 0.3679
evaluating..
1/1             0s 31ms/step
fitting..
Epoch 1/50
1/1             0s 139ms/step -
accuracy: 0.9200 - loss: 0.2384 - val_accuracy: 1.0000 - val_loss: 0.0257
Epoch 2/50
1/1             0s 125ms/step -
accuracy: 0.9600 - loss: 0.1436 - val_accuracy: 1.0000 - val_loss: 0.0255
Epoch 3/50
1/1             0s 165ms/step -
accuracy: 0.9600 - loss: 0.1425 - val_accuracy: 1.0000 - val_loss: 0.0252
Epoch 4/50
1/1             0s 193ms/step -
accuracy: 0.9600 - loss: 0.1419 - val_accuracy: 1.0000 - val_loss: 0.0250
Epoch 5/50
1/1             0s 119ms/step -
accuracy: 0.9600 - loss: 0.1416 - val_accuracy: 1.0000 - val_loss: 0.0248
Epoch 6/50
1/1             0s 121ms/step -
accuracy: 0.9600 - loss: 0.1414 - val_accuracy: 1.0000 - val_loss: 0.0246
Epoch 7/50
1/1             0s 111ms/step -
accuracy: 0.9600 - loss: 0.1412 - val_accuracy: 1.0000 - val_loss: 0.0245
Epoch 8/50
1/1             0s 101ms/step -
accuracy: 0.9600 - loss: 0.1411 - val_accuracy: 1.0000 - val_loss: 0.0244
Epoch 9/50
1/1             0s 102ms/step -
accuracy: 0.9600 - loss: 0.1410 - val_accuracy: 1.0000 - val_loss: 0.0243
Epoch 10/50
1/1             0s 122ms/step -
accuracy: 0.9600 - loss: 0.1410 - val_accuracy: 1.0000 - val_loss: 0.0243
Epoch 11/50
1/1             0s 105ms/step -
accuracy: 0.9200 - loss: 0.2438 - val_accuracy: 1.0000 - val_loss: 0.0246
Epoch 12/50
1/1             0s 150ms/step -
accuracy: 0.9200 - loss: 0.2433 - val_accuracy: 1.0000 - val_loss: 0.0251
Epoch 13/50
1/1             0s 101ms/step -
accuracy: 0.9200 - loss: 0.2422 - val_accuracy: 1.0000 - val_loss: 0.0260
```

```
Epoch 14/50
1/1             0s 106ms/step -
accuracy: 0.9200 - loss: 0.2408 - val_accuracy: 1.0000 - val_loss: 0.0270
Epoch 15/50
1/1             0s 169ms/step -
accuracy: 0.9200 - loss: 0.2391 - val_accuracy: 1.0000 - val_loss: 0.0284
Epoch 16/50
1/1             0s 115ms/step -
accuracy: 0.9200 - loss: 0.2372 - val_accuracy: 1.0000 - val_loss: 0.0299
Epoch 17/50
1/1             0s 116ms/step -
accuracy: 0.9200 - loss: 0.2353 - val_accuracy: 1.0000 - val_loss: 0.0317
Epoch 18/50
1/1             0s 107ms/step -
accuracy: 0.9200 - loss: 0.2333 - val_accuracy: 1.0000 - val_loss: 0.0336
Epoch 19/50
1/1             0s 103ms/step -
accuracy: 0.9200 - loss: 0.2315 - val_accuracy: 1.0000 - val_loss: 0.0357
Epoch 20/50
1/1             0s 104ms/step -
accuracy: 0.9200 - loss: 0.2299 - val_accuracy: 1.0000 - val_loss: 0.0380
Epoch 21/50
1/1             0s 115ms/step -
accuracy: 0.9200 - loss: 0.2285 - val_accuracy: 1.0000 - val_loss: 0.0403
Epoch 22/50
1/1             0s 124ms/step -
accuracy: 0.9200 - loss: 0.2273 - val_accuracy: 1.0000 - val_loss: 0.0428
Epoch 23/50
1/1             0s 112ms/step -
accuracy: 0.9200 - loss: 0.2264 - val_accuracy: 1.0000 - val_loss: 0.0452
Epoch 24/50
1/1             0s 181ms/step -
accuracy: 0.9200 - loss: 0.2258 - val_accuracy: 1.0000 - val_loss: 0.0476
Epoch 25/50
1/1             0s 113ms/step -
accuracy: 0.9200 - loss: 0.2255 - val_accuracy: 1.0000 - val_loss: 0.0499
Epoch 26/50
1/1             0s 113ms/step -
accuracy: 0.9200 - loss: 0.2253 - val_accuracy: 1.0000 - val_loss: 0.0521
Epoch 27/50
1/1             0s 113ms/step -
accuracy: 0.9200 - loss: 0.2253 - val_accuracy: 1.0000 - val_loss: 0.0540
Epoch 28/50
1/1             0s 112ms/step -
accuracy: 0.9200 - loss: 0.2254 - val_accuracy: 1.0000 - val_loss: 0.0557
Epoch 29/50
1/1             0s 171ms/step -
accuracy: 0.9200 - loss: 0.2256 - val_accuracy: 1.0000 - val_loss: 0.0572
```

```
Epoch 30/50
1/1             0s 106ms/step -
accuracy: 0.9200 - loss: 0.2257 - val_accuracy: 1.0000 - val_loss: 0.0583
Epoch 31/50
1/1             0s 144ms/step -
accuracy: 0.9200 - loss: 0.2259 - val_accuracy: 1.0000 - val_loss: 0.0592
Epoch 32/50
1/1             0s 113ms/step -
accuracy: 0.9200 - loss: 0.2260 - val_accuracy: 1.0000 - val_loss: 0.0597
Epoch 33/50
1/1             0s 162ms/step -
accuracy: 0.9200 - loss: 0.2260 - val_accuracy: 1.0000 - val_loss: 0.0600
Epoch 34/50
1/1             0s 113ms/step -
accuracy: 0.9200 - loss: 0.2259 - val_accuracy: 1.0000 - val_loss: 0.0599
Epoch 35/50
1/1             0s 113ms/step -
accuracy: 0.9200 - loss: 0.2258 - val_accuracy: 1.0000 - val_loss: 0.0597
Epoch 36/50
1/1             0s 178ms/step -
accuracy: 0.9200 - loss: 0.2256 - val_accuracy: 1.0000 - val_loss: 0.0592
Epoch 37/50
1/1             0s 120ms/step -
accuracy: 0.9200 - loss: 0.2253 - val_accuracy: 1.0000 - val_loss: 0.0586
Epoch 38/50
1/1             0s 136ms/step -
accuracy: 0.9200 - loss: 0.2250 - val_accuracy: 1.0000 - val_loss: 0.0578
Epoch 39/50
1/1             0s 120ms/step -
accuracy: 0.9200 - loss: 0.2247 - val_accuracy: 1.0000 - val_loss: 0.0569
Epoch 40/50
1/1             0s 130ms/step -
accuracy: 0.9200 - loss: 0.2244 - val_accuracy: 1.0000 - val_loss: 0.0560
Epoch 41/50
1/1             0s 114ms/step -
accuracy: 0.9200 - loss: 0.2241 - val_accuracy: 1.0000 - val_loss: 0.0550
Epoch 42/50
1/1             0s 105ms/step -
accuracy: 0.9200 - loss: 0.2238 - val_accuracy: 1.0000 - val_loss: 0.0540
Epoch 43/50
1/1             0s 112ms/step -
accuracy: 0.9200 - loss: 0.2236 - val_accuracy: 1.0000 - val_loss: 0.0531
Epoch 44/50
1/1             0s 118ms/step -
accuracy: 0.9200 - loss: 0.2233 - val_accuracy: 1.0000 - val_loss: 0.0522
Epoch 45/50
1/1             0s 175ms/step -
accuracy: 0.9200 - loss: 0.2232 - val_accuracy: 1.0000 - val_loss: 0.0513
```

```
Epoch 46/50
1/1             0s 104ms/step -
accuracy: 0.9200 - loss: 0.2230 - val_accuracy: 1.0000 - val_loss: 0.0506
Epoch 47/50
1/1             0s 113ms/step -
accuracy: 0.9200 - loss: 0.2229 - val_accuracy: 1.0000 - val_loss: 0.0499
Epoch 48/50
1/1             0s 111ms/step -
accuracy: 0.9200 - loss: 0.2228 - val_accuracy: 1.0000 - val_loss: 0.0493
Epoch 49/50
1/1             0s 109ms/step -
accuracy: 0.9200 - loss: 0.2227 - val_accuracy: 1.0000 - val_loss: 0.0488
Epoch 50/50
1/1             0s 116ms/step -
accuracy: 0.9200 - loss: 0.2226 - val_accuracy: 1.0000 - val_loss: 0.0485
evaluating..
1/1             0s 40ms/step
fitting..
Epoch 1/50
1/1             0s 147ms/step -
accuracy: 0.9600 - loss: 0.1387 - val_accuracy: 0.8571 - val_loss: 0.3186
Epoch 2/50
1/1             0s 119ms/step -
accuracy: 0.9600 - loss: 0.1382 - val_accuracy: 0.8571 - val_loss: 0.3204
Epoch 3/50
1/1             0s 110ms/step -
accuracy: 0.9600 - loss: 0.1377 - val_accuracy: 0.8571 - val_loss: 0.3227
Epoch 4/50
1/1             0s 136ms/step -
accuracy: 0.9600 - loss: 0.1371 - val_accuracy: 0.8571 - val_loss: 0.3255
Epoch 5/50
1/1             0s 126ms/step -
accuracy: 0.9600 - loss: 0.1364 - val_accuracy: 0.8571 - val_loss: 0.3286
Epoch 6/50
1/1             0s 105ms/step -
accuracy: 0.9600 - loss: 0.1358 - val_accuracy: 0.8571 - val_loss: 0.3320
Epoch 7/50
1/1             0s 100ms/step -
accuracy: 0.9600 - loss: 0.1352 - val_accuracy: 0.8571 - val_loss: 0.3356
Epoch 8/50
1/1             0s 99ms/step -
accuracy: 0.9600 - loss: 0.1346 - val_accuracy: 0.8571 - val_loss: 0.3394
Epoch 9/50
1/1             0s 106ms/step -
accuracy: 0.9600 - loss: 0.1341 - val_accuracy: 0.8571 - val_loss: 0.3432
Epoch 10/50
1/1             0s 103ms/step -
accuracy: 0.9600 - loss: 0.1337 - val_accuracy: 0.8571 - val_loss: 0.3471
```

```
Epoch 11/50
1/1             0s 107ms/step -
accuracy: 0.9600 - loss: 0.1334 - val_accuracy: 0.8571 - val_loss: 0.3509
Epoch 12/50
1/1             0s 104ms/step -
accuracy: 0.9600 - loss: 0.1331 - val_accuracy: 0.8571 - val_loss: 0.3545
Epoch 13/50
1/1             0s 116ms/step -
accuracy: 0.9600 - loss: 0.1329 - val_accuracy: 0.8571 - val_loss: 0.3580
Epoch 14/50
1/1             0s 104ms/step -
accuracy: 0.9600 - loss: 0.1328 - val_accuracy: 0.8571 - val_loss: 0.3613
Epoch 15/50
1/1             0s 140ms/step -
accuracy: 0.9600 - loss: 0.1328 - val_accuracy: 0.8571 - val_loss: 0.3643
Epoch 16/50
1/1             0s 109ms/step -
accuracy: 0.9600 - loss: 0.1327 - val_accuracy: 0.8571 - val_loss: 0.3671
Epoch 17/50
1/1             0s 106ms/step -
accuracy: 0.9600 - loss: 0.1327 - val_accuracy: 0.8571 - val_loss: 0.3695
Epoch 18/50
1/1             0s 108ms/step -
accuracy: 0.9600 - loss: 0.1328 - val_accuracy: 0.8571 - val_loss: 0.3716
Epoch 19/50
1/1             0s 106ms/step -
accuracy: 0.9600 - loss: 0.1328 - val_accuracy: 0.8571 - val_loss: 0.3733
Epoch 20/50
1/1             0s 102ms/step -
accuracy: 0.9600 - loss: 0.1328 - val_accuracy: 0.8571 - val_loss: 0.3748
Epoch 21/50
1/1             0s 99ms/step -
accuracy: 0.9600 - loss: 0.1328 - val_accuracy: 0.8571 - val_loss: 0.3759
Epoch 22/50
1/1             0s 200ms/step -
accuracy: 0.9600 - loss: 0.1329 - val_accuracy: 0.8571 - val_loss: 0.3767
Epoch 23/50
1/1             0s 104ms/step -
accuracy: 0.9600 - loss: 0.1329 - val_accuracy: 0.8571 - val_loss: 0.3772
Epoch 24/50
1/1             0s 118ms/step -
accuracy: 0.9600 - loss: 0.1329 - val_accuracy: 0.8571 - val_loss: 0.3775
Epoch 25/50
1/1             0s 108ms/step -
accuracy: 0.9600 - loss: 0.1329 - val_accuracy: 0.8571 - val_loss: 0.3775
Epoch 26/50
1/1             0s 105ms/step -
accuracy: 0.9600 - loss: 0.1329 - val_accuracy: 0.8571 - val_loss: 0.3772
```

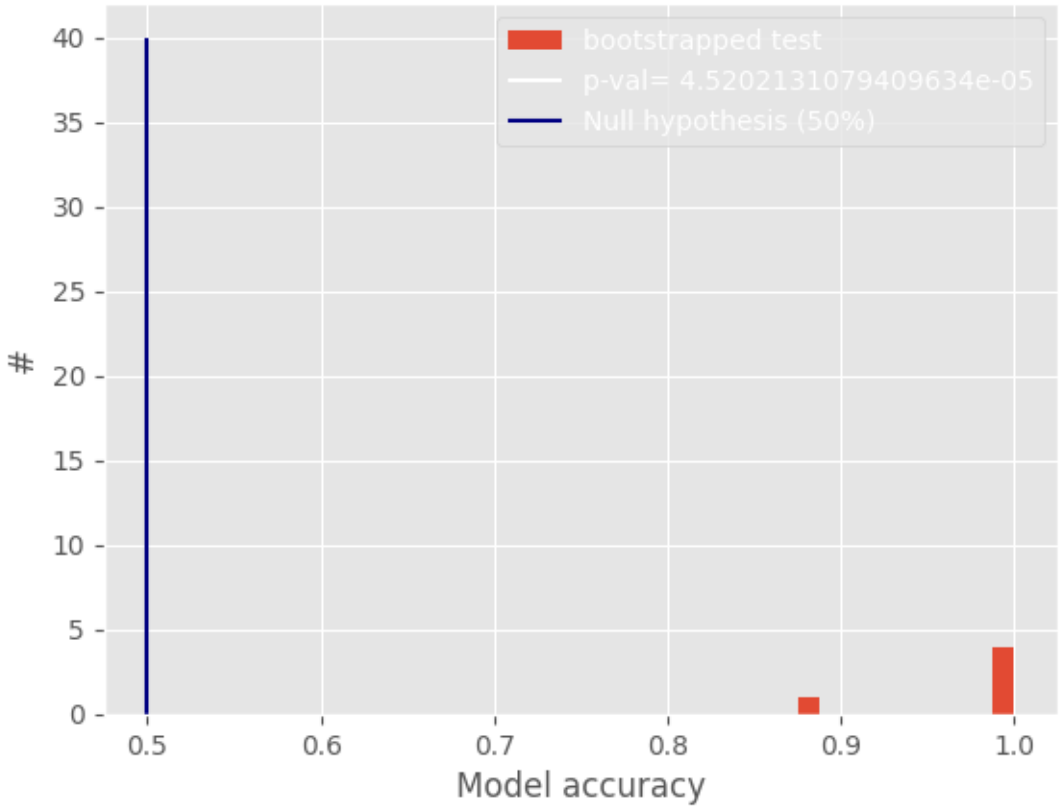
```
Epoch 27/50
1/1             0s 101ms/step -
accuracy: 0.9600 - loss: 0.1328 - val_accuracy: 0.8571 - val_loss: 0.3768
Epoch 28/50
1/1             0s 130ms/step -
accuracy: 0.9600 - loss: 0.1328 - val_accuracy: 0.8571 - val_loss: 0.3762
Epoch 29/50
1/1             0s 113ms/step -
accuracy: 0.9600 - loss: 0.1327 - val_accuracy: 0.8571 - val_loss: 0.3754
Epoch 30/50
1/1             0s 106ms/step -
accuracy: 0.9600 - loss: 0.1327 - val_accuracy: 0.8571 - val_loss: 0.3745
Epoch 31/50
1/1             0s 108ms/step -
accuracy: 0.9600 - loss: 0.1326 - val_accuracy: 0.8571 - val_loss: 0.3735
Epoch 32/50
1/1             0s 104ms/step -
accuracy: 0.9600 - loss: 0.1326 - val_accuracy: 0.8571 - val_loss: 0.3724
Epoch 33/50
1/1             0s 141ms/step -
accuracy: 0.9600 - loss: 0.1325 - val_accuracy: 0.8571 - val_loss: 0.3712
Epoch 34/50
1/1             0s 112ms/step -
accuracy: 0.9600 - loss: 0.1325 - val_accuracy: 0.8571 - val_loss: 0.3701
Epoch 35/50
1/1             0s 105ms/step -
accuracy: 0.9600 - loss: 0.1324 - val_accuracy: 0.8571 - val_loss: 0.3689
Epoch 36/50
1/1             0s 103ms/step -
accuracy: 0.9600 - loss: 0.1324 - val_accuracy: 0.8571 - val_loss: 0.3677
Epoch 37/50
1/1             0s 105ms/step -
accuracy: 0.9600 - loss: 0.1323 - val_accuracy: 0.8571 - val_loss: 0.3666
Epoch 38/50
1/1             0s 104ms/step -
accuracy: 0.9600 - loss: 0.1323 - val_accuracy: 0.8571 - val_loss: 0.3655
Epoch 39/50
1/1             0s 107ms/step -
accuracy: 0.9600 - loss: 0.1322 - val_accuracy: 0.8571 - val_loss: 0.3644
Epoch 40/50
1/1             0s 105ms/step -
accuracy: 0.9600 - loss: 0.1322 - val_accuracy: 0.8571 - val_loss: 0.3634
Epoch 41/50
1/1             0s 106ms/step -
accuracy: 0.9600 - loss: 0.1322 - val_accuracy: 0.8571 - val_loss: 0.3625
Epoch 42/50
1/1             0s 104ms/step -
accuracy: 0.9600 - loss: 0.1322 - val_accuracy: 0.8571 - val_loss: 0.3617
```

```
Epoch 43/50
1/1             0s 133ms/step -
accuracy: 0.9600 - loss: 0.1322 - val_accuracy: 0.8571 - val_loss: 0.3609
Epoch 44/50
1/1             0s 111ms/step -
accuracy: 0.9600 - loss: 0.1321 - val_accuracy: 0.8571 - val_loss: 0.3602
Epoch 45/50
1/1             0s 118ms/step -
accuracy: 0.9600 - loss: 0.1321 - val_accuracy: 0.8571 - val_loss: 0.3597
Epoch 46/50
1/1             0s 100ms/step -
accuracy: 0.9600 - loss: 0.1321 - val_accuracy: 0.8571 - val_loss: 0.3592
Epoch 47/50
1/1             0s 106ms/step -
accuracy: 0.9600 - loss: 0.1321 - val_accuracy: 0.8571 - val_loss: 0.3588
Epoch 48/50
1/1             0s 99ms/step -
accuracy: 0.9600 - loss: 0.1321 - val_accuracy: 0.8571 - val_loss: 0.3585
Epoch 49/50
1/1             0s 106ms/step -
accuracy: 0.9600 - loss: 0.1321 - val_accuracy: 0.8571 - val_loss: 0.3582
Epoch 50/50
1/1             0s 106ms/step -
accuracy: 0.9600 - loss: 0.1321 - val_accuracy: 0.8571 - val_loss: 0.3581
evaluating..
1/1             0s 36ms/step
```

[22]:
```python
#def calc_p_val(stats, h0, n_iterations):
def calc_p_val(stats, h0):
  '''
  finds the p value for the results

  '''
  # calc pval
  tset, pval = ttest_1samp(stats, h0)

  return pval

p_val=calc_p_val(accuracy, .5)
```

[23]:
```python
def plot_p_value(stats, p_val):
  '''
  plots the bootstrapping results with the null hypothesis value
  '''

  plt.hist(stats, label='bootstrapped test')
  plt.vlines(.5, 0, 40, color='white', label='p-val= {}'.format(p_val))
```

```
  plt.vlines(.5, 0, 40, color='navy', label='Null hypothesis (50%)')

  plt.title('Histogram model accuracy bootstrapping')
  plt.xlabel('Model accuracy')
  plt.ylabel('#')
  plt.legend()
  plt.plot()

plot_p_value(accuracy, p_val)
```



[24]:
```
def plot_roc_curve(fpr_vals, tpr_vals, roc_auc, p_val):
  '''
  This function plots the median value of the roc for the boostrapped
    results calculated above.

  fpr stand for false-positive rate
  tpr stands for true-positive rate
  roc_auc is the area under curve
  '''
```

```
  ## get the values
  N=len(fpr_vals)
  tprs=[]
  median_fpr=np.linspace(0, 1, 100)
#  tprs=[interp(median_fpr, fpr_vals[i], tpr_vals[i]) for i in range(N)]
  tprs=[np.interp(median_fpr, fpr_vals[i], tpr_vals[i]) for i in range(N)]
  std_tpr = np.std(tprs, axis=0)

  mean_tpr = np.mean(tprs, axis=0)
  median_tpr=np.median(tprs, axis=0)
  median_tpr[-1] = 1.0

  tprs_upper_2 = np.minimum(mean_tpr + 2*std_tpr, 1)
  tprs_lower_2 = np.maximum(mean_tpr - 2*std_tpr, 0)

  tprs_upper_1 = np.minimum(mean_tpr + std_tpr, 1)
  tprs_lower_1 = np.maximum(mean_tpr - std_tpr, 0)

  median_auc_roc=np.median(roc_auc)


  ## plot
  if p_val<0.05:
    p_val=0.05
  plt.plot(median_fpr, median_tpr, color='cadetblue',
           label='ROC curve \narea={} \np-val<{}'.\
            format(np.round(median_auc_roc,2),
                   np.round(p_val,2)))
  plt.fill_between(median_fpr, tprs_lower_2, tprs_upper_2, color='grey', alpha=.
 ↪2,
                  label=r'$\pm$ 1 std. dev.')

  plt.fill_between(median_fpr, tprs_lower_1, tprs_upper_1, color='cadetblue',␣
 ↪alpha=.2,
                  label=r'$\pm$ 2 std. dev.')

  plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label=r'chance')

  plt.xlabel('False Positive Rate')
  plt.ylabel('True Positive Rate')
  plt.title('Receiver operating characteristic curve')
  plt.legend(loc="lower right")

  plt.show()
```

```
plot_roc_curve(roc_msrmnts_fpr, roc_msrmnts_tpr, accuracy,p_val)
```